

Componente Teórico

Prueba técnica

Pregunta 1. Ciclo de vida de un modelo

Entendimiento del negocio: definir el problema con el área comercial. No es "quiero un modelo", es "necesito anticipar qué clientes se van para actuar antes". De acá sale la variable objetivo, la ventana de tiempo y las métricas relevantes.

Exploración y preparación de datos: revisar fuentes, tratar nulos, duplicados y outliers. Cruzar tablas de clientes, transacciones y productos. Feature engineering con variables que capturen comportamiento real.

Modelado y validación: probar al menos un modelo interpretable (logística) y uno de mayor capacidad (Random Forest). Split estratificado train/val/test, evaluar con AUC, recall y KS — no con accuracy, porque el desbalance la infla.

Aprobación y despliegue: validar estabilidad (PSI), documentar supuestos, presentar al negocio. Empaquetar el modelo y exponerlo como API o proceso batch que alimente el CRM con scores.

Post-producción:

- *Monitoreo:* revisar métricas y PSI mensualmente sobre variables de entrada y score.
- *Drift:* si la población cambia, el modelo se degrada. $PSI > 0.25$ es señal de alerta.
- *Reentrenamiento:* periódico o por alerta de drift. Solo se reemplaza si el nuevo modelo supera al vigente.
- *Gobierno:* versionamiento, registro de aprobaciones, datos de entrenamiento y auditoría de sesgo. En una financiera esto es regulatorio.

Pregunta 2. Conceptos esenciales

Variable objetivo: lo que el modelo intenta predecir. En churn, es una columna binaria (1 = se fue, 0 = se quedó).

Ventana de observación: período histórico del que se extraen las variables predictoras. Ej: últimos 6 meses de transacciones y comportamiento.

Ventana de desempeño: período futuro en el que se observa si el evento ocurrió. Ej: los 3 meses siguientes para ver si el cliente abandonó. La separación entre ambas ventanas evita usar información del futuro.

Train / Validación / Test: tres particiones. Train (70%) para aprender, validación (15%) para afinar hiperparámetros, test (15%) para evaluación final. Se estratifican para mantener la proporción de churn.

Validación cruzada: se divide train en K pliegues (típicamente 5), se entrena K veces dejando uno como validación. Resultado más robusto que un solo split. Útil con datasets pequeños.

Data leakage: cuando el modelo accede a información que no tendría al momento de predecir. Ej: escalar con la media de todo el dataset incluyendo test, o usar variables posteriores al evento.

Desbalanceo de clases: cuando una clase domina (~80% no-churn vs ~20% churn). El modelo puede aprender a decir "nadie se va" y tener 80% accuracy sin detectar nada. Se compensa con `class_weight='balanced'` o métricas como recall y AUC.

Feature engineering: crear variables nuevas que capturen mejor los patrones. Ej: `BalanceSalaryRatio` o `NivelVinculacion` desde reglas de negocio en JSON. Suele impactar más que la elección del algoritmo.

Pregunta 3. Métricas en clasificación desbalanceada

De la matriz: TN=70, FP=10, FN=15, TP=5.

Precision = $TP/(TP+FP) = 5/15 = 0.333$ — de los marcados como churn, solo 1 de cada 3 realmente se fue.

Recall = $TP/(TP+FN) = 5/20 = 0.250$ — de los 20 que se fueron, el modelo solo detectó 5.

Accuracy = $(TP+TN)/Total = 75/100 = 0.750$ — engañosa: decir "nadie se va" daría 80%.

F1-score = $2 \times (P \times R) / (P + R) = 0.286$

Priorizaría Recall. Cada cliente no detectado es uno que se va sin contacto. Reemplazarlo cuesta mucho más que una campaña de retención innecesaria (falso positivo). Con recall de 0.25 el modelo pierde 15 de cada 20 clientes en riesgo. Habría que bajar el umbral o rebalancear las clases.

Pregunta 4. Selección de modelo

Modelo B (Random Forest). Razones:

Estabilidad: brecha train/test menor en todos los indicadores (KS: 30→28%, accuracy: 70→69%, recall: 60→58%). A pierde 11 puntos de KS → sobreajuste. C sube accuracy en test (70→80%), sospechoso.

PSI: B tiene 9% (estable). C tiene 17%, zona de alerta.

Recall: B detecta 58% de churners en test, vs 38% de A y 35% de C. Para retención es la métrica clave.

Interpretabilidad: con SHAP e importancia de variables es suficiente para auditoría. La red neuronal es la más opaca ante un comité de riesgo.

Pregunta 5. SQL aplicado

Monto total transado por cliente en 2025:

```
SELECT id_cliente, SUM(monto) AS monto_total
FROM transacciones
WHERE fecha >= '2025-01-01' AND fecha < '2026-01-01'
GROUP BY id_cliente
ORDER BY monto_total DESC;
```

Top 5 ciudades por clientes activos (transacción en últimos 90 días):

```
SELECT c.ciudad, COUNT(DISTINCT c.id_cliente) AS clientes_activos
FROM clientes c
INNER JOIN transacciones t ON c.id_cliente = t.id_cliente
WHERE t.fecha >= DATEADD(DAY, -90, GETDATE())
GROUP BY c.ciudad
ORDER BY clientes_activos DESC LIMIT 5;
```

Monto promedio mensual por segmento en 2025:

```
SELECT c.segmento, MONTH(t.fecha) AS mes, AVG(t.monto) AS monto_promedio
FROM clientes c
INNER JOIN transacciones t ON c.id_cliente = t.id_cliente
WHERE t.fecha >= '2025-01-01' AND t.fecha < '2026-01-01'
GROUP BY c.segmento, MONTH(t.fecha)
ORDER BY c.segmento, mes;
```

Última oportunidad por cliente (función de ventana):

```
SELECT id_cliente, id_oportunidad, fecha_creacion, estado, score
FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY id_cliente
        ORDER BY fecha_creacion DESC) AS rn
    FROM oportunidades
) sub WHERE rn = 1;
```

INNER JOIN vs LEFT JOIN:

INNER JOIN cuando solo interesan clientes con registros en ambas tablas (ej: clientes que transaron).
 LEFT JOIN cuando necesito conservar todos los clientes aunque no tengan transacciones — útil para identificar inactivos (candidatos a churn) filtrando con WHERE t.id_transaccion IS NULL.

Pregunta 6. Python y limpieza de datos

¿Qué hace el fragmento?

Carga un CSV, imputa nulos de monto con la mediana, calcula media y conteo por segmento, y filtra registros por debajo del percentil 99 para eliminar outliers.

Problema metodológico:

El orden está invertido. Imputa nulos con la mediana inflada por outliers, calcula el resumen sobre datos aún contaminados, y recién al final elimina outliers. Lo correcto: (1) eliminar outliers, (2) imputar con la mediana limpia, (3) calcular el resumen. Además, la imputación debería hacerse por segmento (groupby.transform) porque el monto típico varía entre segmentos.

Función reutilizable:

```
def limpiar_columna(df, columna, grupo=None, limite_iqr=1.5):
    df = df.copy()
    Q1 = df[columna].quantile(0.25)
    Q3 = df[columna].quantile(0.75)
    IQR = Q3 - Q1
    df[columna] = df[columna].clip(Q1 - limite_iqr*IQR, Q3 + limite_iqr*IQR)
    if grupo and grupo in df.columns:
        mediana = df.groupby(grupo)[columna].transform('median')
        df[columna] = df[columna].fillna(mediana)
    else:
```

```
df[columna] = df[columna].fillna(df[columna].median())  
return df
```

Usa clip en vez de eliminar filas para no perder registros. Permite segmentar la imputación por grupo.

Pregunta 7. APIs, JSON y Git

Consumo de API REST:

Autenticarse (Bearer token en headers, credenciales en variables de entorno), hacer requests paginados iterando hasta respuesta vacía, parsear JSON de cada página, acumular en lista y convertir a DataFrame con `pd.json_normalize`. Manejar rate limits (respetar header `Retry-After`, sleep preventivo entre requests), errores (try/except con logging de contexto) y validar el DataFrame resultante (columnas esperadas, tipos correctos, sin duplicados).

Flujo Git:

Feature branches: main protegida, nadie hace push directo. Para cada tarea se crea rama (feature/modelo-churn), commits frecuentes con mensajes descriptivos, Pull Request con code review de al menos un compañero, resolver conflictos en la rama feature antes del merge, eliminar rama después. Archivos pesados (.csv, .pkl) en .gitignore; si se necesita versionarlos, usar DVC o Git LFS.

Pregunta 8. IA generativa, RAG y uso seguro

Comparación:

Prompt engineering: adaptar el LLM vía instrucciones en el prompt. Rápido e iterativo, pero el modelo no conoce documentos internos.

RAG: buscar fragmentos relevantes en una base documental vectorizada e inyectarlos en el contexto. Se actualiza sin reentrenar, cada respuesta es trazable al documento fuente.

Fine-tuning: reentrenar con datos propios. Respuestas más naturales pero costoso, cada actualización requiere reentrenamiento, y riesgo de memorizar datos sensibles.

Para un asistente de políticas internas usaría RAG: las políticas cambian frecuentemente (solo actualizo el índice), las respuestas son trazables al documento fuente (auditoría), y el riesgo de alucinación es menor al condicionar las respuestas a los fragmentos recuperados. Complementaría con prompt engineering en el system prompt.

Mitigación de riesgos:

Alucinaciones: instruir al modelo para responder solo desde documentos recuperados y decir "no tengo información" si no encuentra respaldo. Validar coherencia respuesta vs fragmentos citados.

Datos sensibles: clasificar documentos por nivel de acceso, el retriever solo busca en documentos permitidos. No indexar datos personales. Anonimizar en el pipeline de ingesta. No loguear datos de clientes.

Prompt injection: tratar contenido recuperado como dato, nunca como instrucción. Separar system prompt de documentos. Filtrar patrones como "ignora las instrucciones anteriores" antes de pasarlos al modelo. Validar salidas contra reglas de negocio.